# 문제해결 프로그래밍 기말텀 보고서

1940200_임재현

## 주제: 그림으로 감정(심리적 상태) 예측하기

감정 표현의 수단으로서 언어는 구체적이고 표현이 명확하지만 정작 감정이라는 것은 추상적 개념에 불과하여 정확한 감정에 대한 언어적 표현이 모호할 수 있음
따라서 감정을 표현한 매체로서 추상성을 표현할 수 있는, 그림을 분석하여 해당 그림이 감정과 매핑 되도록 하는 것을 목표로 함

## 프로젝트 설계 및 진행 과정(흐름)

1. 그림(이하 이미지라고 함)에서 추출 할 수 있는 특성은 어떤 것이 있는가?
   => 이미지에서의 특성 종류 파악하기
   => 이미지에서 추출 가능한 특성은 색상, 선의 곡률, 이미지를 그린 도구가 나타내는 선의 질감(붓, 연필, 볼펜 등) 등이 있음

2. 사람의 얼굴 표정이나 행동을 CNN 모델에 학습 시킨다면 그에 알맞는 특성이 추출 가능하겠지만, 이미지는 꼭 그런 특성이 돋보이는 요소만을 그리는게 아닌, 추상적인 것(자유로운 낙서 형식의 그림)을 그릴 수도 있음. 또한 이미지가 나타내는 감정을 레이블링 하기 어려움(말 그대로 감정과 그림 모두 추상적 개념이기 때문임).

따라서 선 특성을 학습 시키는 것은 레이블링을 하는데 주관성이 들어갈 가능성이 있고, 그림의 질감 등을 레이블링 하는데에 있어서 데이터 수집에 어려움이 있기 때문에, 색상 특성을 기준으로 하여 이미지가 나타내는 감정을 예측 하는 모델을 만듦.

3. 대신 그림을 그릴 때에는 그림을 그린 사람이 강조하고 싶은 부분이 있다고 생각함. 따라서 선의 특성 추출이 힘들다면 그림에서 강조하는 영역을 추출하여 해당 영역이 사용하는 주요 색상이 무었인지를 추출하여 분석하면 감정 예측에 더욱 도움이 될 것이라 생각됨
   => 이미지에서 현저한 영역을 분리하여 해당 영역의 색상 특성 위주로 감정 학습 및 예측에 힘을 실어주기

4. 평가 항목을 추가하여 사용자의 주관이 들어간 데이터를 수집 하여 일반화를 시키도록 하기

# 데이터 수집

색상과 감정 데이터를 수집 하기 위해 Kaggle에서 Color associations in Art: Pt 1 Warmth를 사용함

## Color associations in Art: Pt 1 Warmth

V1 - Basics to train an AI the connections of art and emotion - #CLASSIFY WARMTH

해당 데이터 파일은총 33가지의 색상에 대한 R, G, B 값과 그 색상에 연관된 감정을 3가지로 나타내고 있음

colorassociations_warmth - colorwarmth

| | COLOR_ID | HEX_CODE | RGB_ID | CMYK_ID | HSV_ID | WARM | ASSOCIATION_1 | ASSOCIATION_2 | ASSOCIATION_3 |
|---|---|---|---|---|---|---|---|---|---|
| | RED | #961a1a | 150, 26, 26 | 0%, 83%, 83%, 41% | 0°, 83%, 59% | 1 | INTENSITY | RAGE | ROMANCE |
| | PURPLE | #811a96 | 129, 26, 150 | 14%, 83%, 0%, 41% | 290°, 83%, 59% | 0 | CREATIVITY | ROYALTY | MYSTERY |
| | ORANGE | #ed7428 | 237, 116, 40 | 0%, 51%, 83%, 7% | 23°, 83%, 93% | 1 | ENERGY | ALERTNESS | FRIENDSHIP |
| | GREEN | #139611 | 19, 150, 17 | 87%, 0%, 89%, 41% | 119°, 89%, 59% | 0 | HARMONY | FRESHNESS | IMMATURITY |
| | DARK BLUE | #191196 | 25, 17, 150 | 83%, 89%, 0%, 41% | 244°, 89%, 59% | 0 | AUTHORITY | ELEGANCE | DEPTH |

색상에 해당되는 세개의 감정이 서로 잘 매칭 되지 않는 경우가 있으므로, 색상 하나 당 감정 하나를 매핑하기 위해 중복되는 감정이 있더라도 각 감정들이 가질 수 있는 색상들을 모두 분리 하였음.

이 프로그램의 동작 방식은
**이미지 입력 -> 색상 추출 -> 색상에 따라 감정 예측**
이므로 사용자가 그림을 그린 후 본인이 의도한 색상이 감정 하나에 매핑이 되도록 감정-색상의 형식으로 데이터를 분리하고, 겹치는 색상(동일한 색상)이 없도록 RGB값을 임의로 조정함

color_emotion_labeled_updated

| emotion | R | G | B | color_name | color_label |
|---|---|---|---|---|---|
| **INTENSITY** | 150 | 36 | 26 | RED | 1.0 |
| **INTENSITY** | 220 | 30 | 60 | CRIMSON RED | 2.0 |
| **RAGE** | 151 | 19 | 16 | RED | 3.0 |
| **ROMANCE** | 141 | 30 | 16 | RED | 4.0 |

```python
# RGB 값 조정 함수
def adjust_rgb(r, g, b, emotion):
    # 감정 이름을 해시값으로 변환하여 고유한 오프셋 생성 (0-1000 범위로 조정)
    emotion_hash = abs(hash(emotion)) % 1000
    np.random.seed(emotion_hash)  # 같은 감정은 항상 같은 오프셋을 가지도록

    # RGB 각 채널에 대해 -10 ~ +10 사이의 랜덤한 오프셋 적용
    r_offset = np.random.randint(-10, 11)
    g_offset = np.random.randint(-10, 11)
    b_offset = np.random.randint(-10, 11)

    # RGB 값 조정 (0-255 범위 유지)
    r = max(0, min(255, r + r_offset))
    g = max(0, min(255, g + g_offset))
    b = max(0, min(255, b + b_offset))

    return r, g, b
```

```python
# 감정별로 RGB 값 조정
adjusted_rows = []
for emotion in df['emotion'].unique():
    emotion_data = df[df['emotion'] == emotion]
    for _, row in emotion_data.iterrows():
        r, g, b = adjust_rgb(row['R'], row['G'], row['B'], emotion)
        adjusted_rows.append({
            'emotion': emotion,
            'R': r,
            'G': g,
            'B': b,
            'color_name': row['color_name']
        })
```

^데이터 전처리 하는코드 중 일부(이 자료를 제출 하려면 다른 모든 자료를 제출해야 하기 때문에 이 코드는 제출 자료에 포함되어 있지 않습니다)

```csv
emotion,R,G,B,color_name
INTENSITY,150,26,26,RED
RAGE,150,26,26,RED
ROMANCE,150,26,26,RED
CREATIVITY,129,26,150,PURPLE
ROYALTY,129,26,150,PURPLE
MYSTERY,129,26,150,PURPLE
ENERGY,237,116,40,ORANGE
ALERTNESS,237,116,40,ORANGE
FRIENDSHIP,237,116,40,ORANGE
HARMONY,19,150,17,GREEN
FRESHNESS,19,150,17,GREEN
IMMATURITY,19,150,17,GREEN
AUTHORITY,25,17,150,DARK BLUE
```

```csv
emotion,R,G,B,color_name
INTENSITY,146,27,35,RED
INTENSITY,216,21,69,CRIMSON RED
RAGE,147,16,26,RED
ROMANCE,143,35,32,RED
CREATIVITY,137,34,1
ROYALTY,130,34,157,
MYSTERY,119,35,149,PURPLE
ENERGY,245,106,35,ORANGE
ENERGY,255,26,0,SCARLET
ENERGY,186,24,29,FIREBRICK
ENERGY,186,24,29,FIREBRICK
ALERTNESS,241,115,44,ORANGE
FRIENDSHIP,236,109,49,ORANGE
```

Col 5: color_r

# 학습 방법 및 결과 도출 방법

1. 전처리 데이터(사용할 기본 데이터) 학습 시키기

```
from sklearn.neighbors import KNeighborsClassifier
```

```
# RGB 정규화
color_map['R_norm'] = color_map['R'] / 255
color_map['G_norm'] = color_map['G'] / 255
color_map['B_norm'] = color_map['B'] / 255

# KNeighborsClassifier 학습용 데이터
X = color_map[['R_norm', 'G_norm', 'B_norm']]
y = color_map['emotion']

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X, y)
```

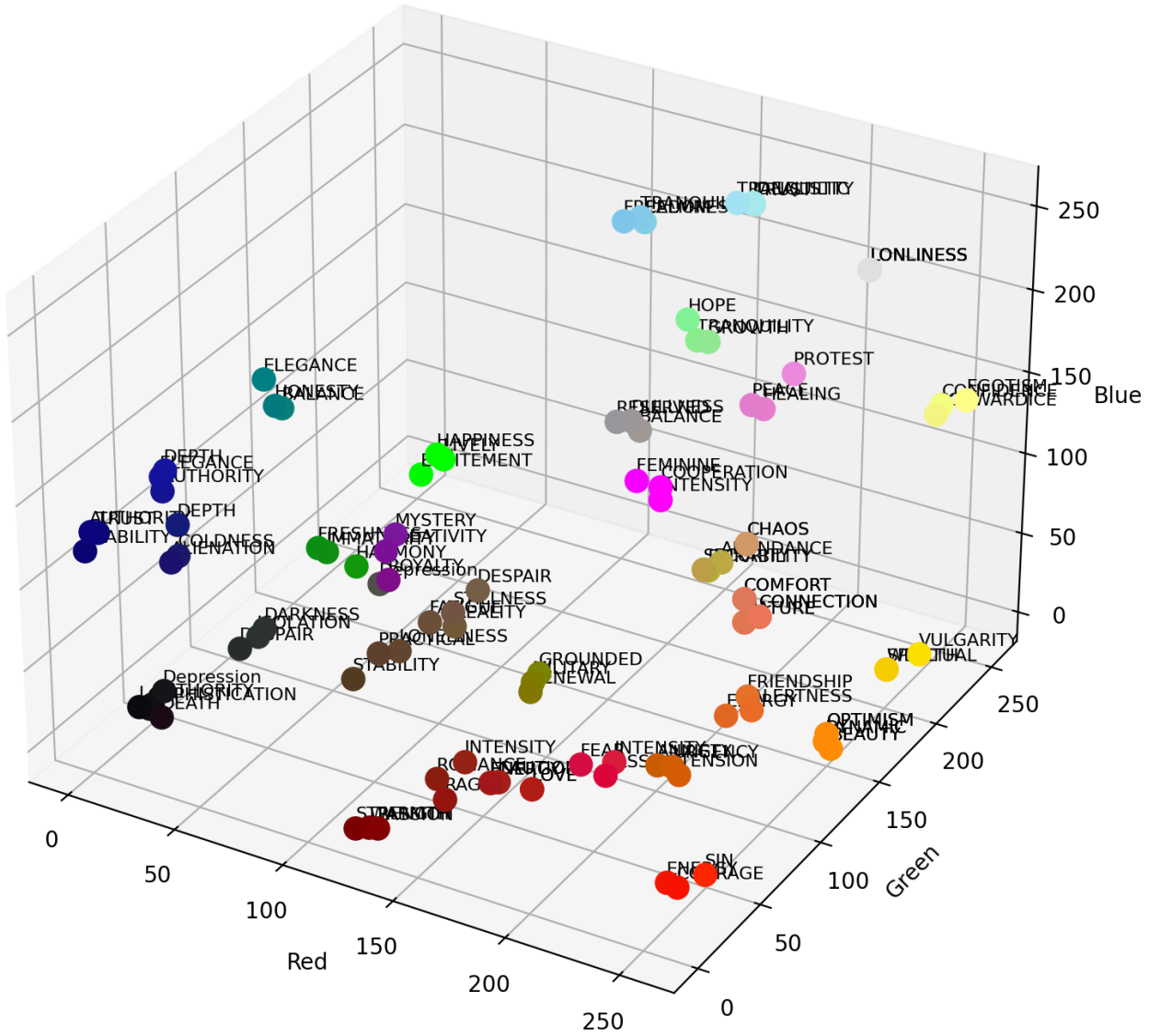
sklearn의 KNeighborsClassifier을 사용하여 전처리한 데이터의 R, G, B 값들을 x값으로 그에 알맞는 감정 y 값에 학습 시켜 예측 결과를 가져오도록 함

```
main_

       R_norm    G_norm    B_norm actual_emotion predicted_emotion
0    0.588235  0.141176  0.101961      INTENSITY           CAUTION
1    0.862745  0.117647  0.235294      INTENSITY              FEAR
2    0.592157  0.074510  0.062745           RAGE         INTENSITY
3    0.552941  0.117647  0.062745        ROMANCE         INTENSITY
4    0.490196  0.078431  0.603922     CREATIVITY        CREATIVITY
..        ...       ...       ...            ...               ...
104  0.882353  0.882353  0.882353      LONLINESS         LONLINESS
```

```
[109 rows x 5 columns]
자체 데이터 정확도: 0.47706422018348627
```

학습 결과 이미지화




보고서에 사용될 입력 데이터

2. 데이터 입력 및 특성 추출

```
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img_resized = cv2.resize(img, dsize: (100, 100))

# 현저성
gray = cv2.cvtColor(img_resized, cv2.COLOR_RGB2GRAY)
blurred = cv2.GaussianBlur(gray, ksize: (5, 5), sigmaX: 0)
laplacian = cv2.Laplacian(blurred, cv2.CV_64F)
saliencyMap = np.uint8(np.absolute(laplacian))
_, salient_mask = cv2.threshold(saliencyMap, thresh: 10, maxval: 255, cv2.THRESH_BINARY)

salient_pixels = img_resized[salient_mask == 255].reshape(-1, 3)
```

^입력 이미지를 변수 img에 선언하고 입력 받은 이미지에서 현저성을 분리하는 OpenCV의 현저성 분리 코드

1) gray 변수에 있는 cv2.COLOR_RGB2GRAY를 통해 입력 이미지의 색상 정보를 밝기 정보로 바꿈
=> 그림에서 윤곽선 부분을 그저 다른 색상을 사용했음으로 구분 하는 것이 아닌 색상의 밝기로 판단하여 윤곽선을 구분하기 위해 사용함

2) blurred를 통해 입력 이미지의 노이즈를 제거하여 윤곽 판단을 더욱 분명하게 하도록함
=> 윤곽에 가까운 부분에 가중치를 더욱 실어주고 윤관에서 먼 부분은 윤곽선과 다른 영역임을 나타내기 위함임

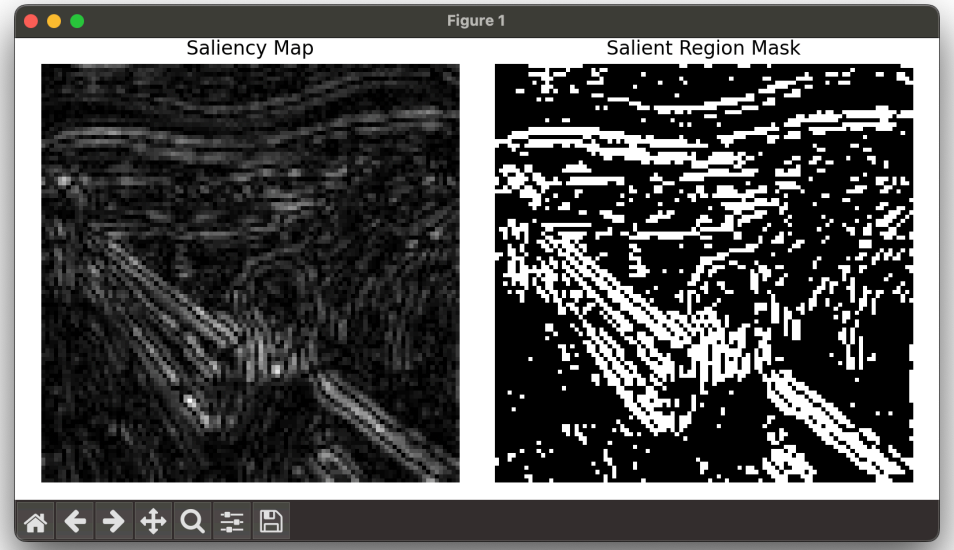
3) laplacian을 이용하여 밝기가 급변하는 곳을 감지하여 경계선을 더욱 정확히 추출하게 함

4) 현저성 맵을 시각화 하기

5) 임계값 10을 기준으로(밝기 기준) 하여 밝기 변화가 10 이상이면 눈에 띄는 것으로 간주하고 이때 색상 값을 255로 주어 흰색으로 나타나게 함

[컬러 이미지] -> 1)(RGB → Gray) -> [흑백 이미지] -> 2)(GaussianBlur) -> [부드러운 흑백 이미지] -> 3)(Laplacian) -> [에지 강조 이미지] -> 4)(abs + uint8) -> [밝기 변화 강도 맵] -> 5)(Threshold)
=> [관심 영역 마스크 생성]

결과 =>



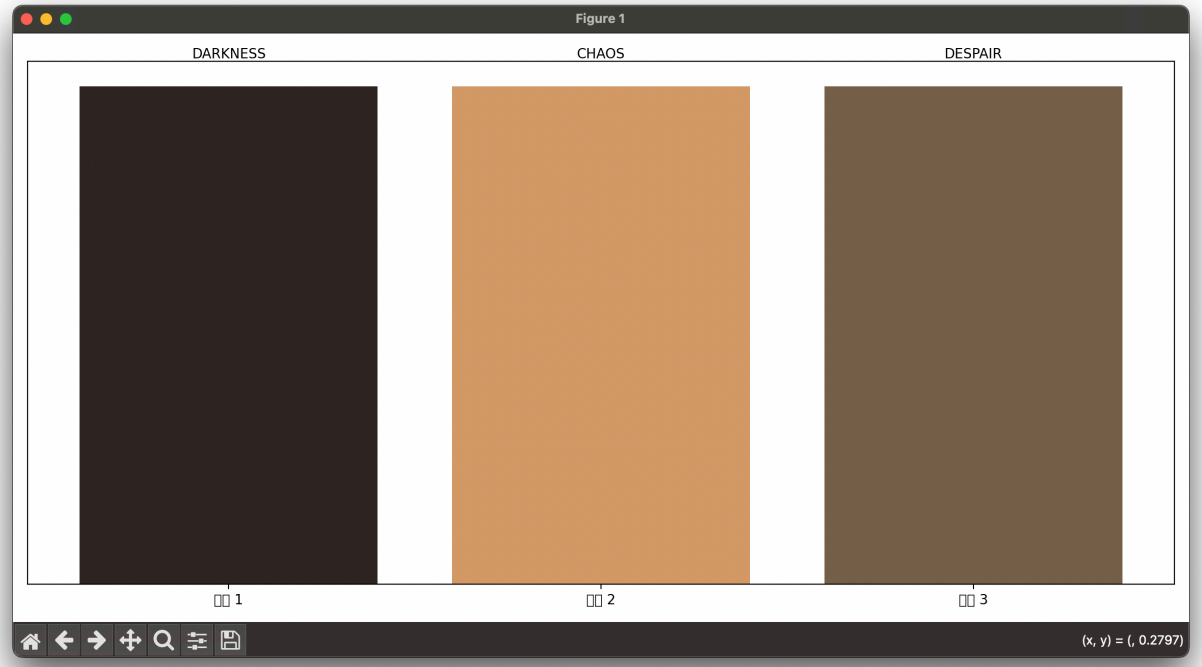
3. 입력 데이터 분석

```
from sklearn.cluster import KMeans
```

```
# KMeans 클러스터링으로 주요 색상 추출
k = 3
kmeans = KMeans(n_clusters=k, random_state=42, n_init='auto')
kmeans.fit(salient_pixels)
dominant_colors = kmeans.cluster_centers_

print("\n[saliency] Extracted Color-Emotions:")
for i, color in enumerate(dominant_colors):
    r, g, b = color
    # 정규화
    color_norm = np.array([[r/255, g/255, b/255]])
    # 감정 예측
    predicted_emotion = knn.predict(color_norm)[0]
    print(f"color {i+1}: RGB = ({int(r)}, {int(g)}, {int(b)}) | 감정 예측 = {predicted_emotion}")
```

sklearn의 KMeans를 사용하여 현저성 맵에서 추출된 색상 중 대표 색상 3개를 가져와, 각 색상이 csv 파일에 있는 색상 중 어느 색상에 가장 가까운지를 판단하여, 추출된 3가지 색상의 각 각 감정을 KNeiborsClassifier에 다시 학습 시켜 예측된 감정 결과를 출력함




```
[saliency] Extracted Color-Emotions:
color 1: RGB = (45, 38, 33) | 감정 예측 = DARKNESS
color 2: RGB = (211, 152, 102) | 감정 예측 = CHAOS
color 3: RGB = (120, 97, 75) | 감정 예측 = DESPAIR
```

4. 평가

사용자가 색상에 의도를 넣어 그림을 그렸는데, 예측된 결과가 사용자가 의도한 색상과 감정이 다르게 매핑 되어 있다면 평가를 통해 감정 데이터를 추가로 수집 할 수 있도록 함

```
[1] 예측된 색상 RGB = (45, 38, 33) → 예측 감정: DARKNESS
이 감정이 맞습니까? (Enter = 예, 아니면 원하는 감정 입력): y
기존 예측 감정 유지, CSV에 추가하지 않습니다.

[2] 예측된 색상 RGB = (211, 152, 102) → 예측 감정: CHAOS
이 감정이 맞습니까? (Enter = 예, 아니면 원하는 감정 입력): /Lib
  warnings.warn(
y
기존 예측 감정 유지, CSV에 추가하지 않습니다.

[3] 예측된 색상 RGB = (120, 97, 75) → 예측 감정: DESPAIR
이 감정이 맞습니까? (Enter = 예, 아니면 원하는 감정 입력): /Lib
  warnings.warn(
SAD

 감정 데이터가 성공적으로 추가되었습니다.
```

| | | | | | |
|---|---|---|---|---|---|
| **DESPAIR** | 120 | 97 | 75 | | |
| **LONLINESS** | 225 | 225 | 225 | | |
| **LONLINESS** | 225 | 225 | 225 | | |
| **LONLINESS** | 225 | 225 | 225 | | |
| **LONLINESS** | 225 | 225 | 225 | | |
| **CHAOS** | 211 | 152 | 102 | | |
| **CHAOS** | 211 | 152 | 102 | | |
| **SAD** | 120 | 97 | 75 | | |

위 처럼 동일한 색상에 대해 동일한 감정이 많이 매핑 되어 있으면 다시 KneighborsClassifier을 수행 할 때, 동일 색상 데이터에 대한 동일 감정 값에 더욱 일관성이 높아져, 예측에 확신이 높아질 것으로 예상됨.
다라서 수집되는 데이터가 많아질 수록 색상-감정 데이터에 일반화를 할 수 있는 확률이 높아질 것으로 생각됨.

# 개선 사항 & 코멘트(프로그램 확장, 상업적 활용 방안 등)

1. 이미지에 대한 감정을 일반화 하는 것은 매우 어려운 일 인것 같음.

특히 색상 만으로는 그림을 그린 어떤 색상으로 그림을 그렸느냐 보다는 무었을 그렸느냐가 사용자의 감정을 판단하는데 더욱 중요한 요소가 될 것 같음.

이 프로젝트에서 추가할 수 있는 부분이 있다면 현저성을 분리하는 코드를 좀 더 연구해서 선 특성을 분리할 수 있지 않을까 라는 생각이 듦.
CNN을 통해 분리된 선 특성을 입력 이미지로 하고 학습 이미지를 일반적인 사물이나 사람의 표정 등으로 하여 선 특성과 학습 이미지가 매칭 되게 한 뒤, 해당 학습된 이미지가 어느 감정을 나타내는지를 레이블링 되게 하면 어떨까 하는 생각이 듦.

하지만 보통 감정을 드러내는 그림을 인터넷에 찾아보면 대부분이 추상화인 경우가 많음. 따라서 선 특성을 표정 등에 매칭 하는 부분에서 부터 매우 힘든 작업이 될 것이라 생각함.

2. 이 프로그램을 어플리케이션화 하기 위해 사용자가 인터페이스에서 직접 그림을 그리는 기능을 추가하면 될좋을것 같음.

사진을 업로드 하는 것은 불가능 하게 만들고, 사용자 본인이 그린 그림만을 다른 사람들에게 자랑하도록 커뮤니티 시스템을 만들어서 사람들과 소통 하도록 하면, 익명성은 보장 하지만 사용자 개개인 만의 개성을 나타내면서 소통할수 있는 기능을 구현해 보면 좋을듯 함.